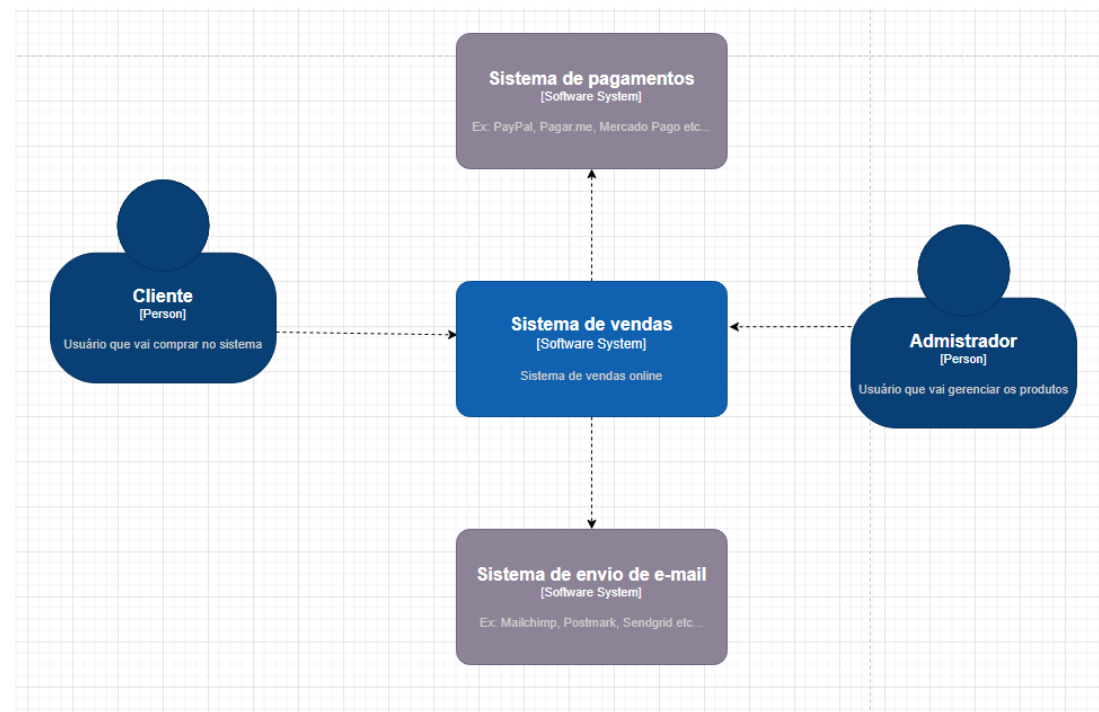


Diagrama de Contexto: Conceito e Exemplos de Uso

O **diagrama de contexto** é o nível mais alto do **modelo C4** e tem como função fornecer uma **visão geral** do sistema como um todo, mostrando suas **interações com o ambiente externo**. Ele ajuda a responder perguntas como:

- O que o sistema faz?
- Quem interage com o sistema?
- Quais são os atores e sistemas externos e como eles se conectam ao sistema?

No **diagrama de contexto**, o sistema é representado como **uma única caixa**, geralmente **centralizada**, enquanto os **atores externos** (usuários, outros sistemas, bancos de dados, APIs, etc.) são colocados ao redor, conectados por **linhas que representam a comunicação e o fluxo de dados**.



Exemplos de Uso do Diagrama de Contexto

1 Exemplo: Sistema de Gestão Escolar

No caso de um sistema de **gestão escolar**, um diagrama de contexto pode ser representado assim:

Sistema: Sistema de Gestão Escolar

Atores Externos:

Aluno (acessa notas e presença)

Professor (lança notas e frequência)

Secretaria (faz matrículas e emite documentos)

Banco (processa pagamentos de mensalidades)

No diagrama de contexto, cada ator estaria conectado ao **Sistema de Gestão Escolar** com suas respectivas interações.

Microkernel

É um modelo de sistema operacional onde apenas as funções essenciais ficam no núcleo, enquanto outros serviços rodam em espaço de usuário e se comunicam por mensagens.

Principais Características do Microkernel

- Kernel mínimo: Apenas funcionalidades básicas, como gerenciamento de processos e comunicação entre processos.
- Serviços no espaço do usuário: Drivers, sistema de arquivos e outros componentes funcionam separadamente do kernel.
- Comunicação via mensagens: O sistema usa troca de mensagens entre os componentes, melhorando modularidade.
- Maior segurança e estabilidade: Como os serviços rodam separados do kernel, falhas afetam apenas partes do sistema, não o núcleo inteiro.
- Desempenho pode ser menor: O uso de mensagens para comunicação pode introduzir uma leve perda de desempenho comparado a um kernel monolítico.

Interface Gráfica no Padrão MVC

Controller + View = Interface Gráfica

O MVC (Model-View-Controller) é um padrão de arquitetura amplamente utilizado no desenvolvimento de sistemas.

Ele divide a aplicação em três camadas principais:

- ✓ Model (Modelo) – Representa os dados e a lógica da aplicação.
- ✓ View (Vista) – Responsável pela exibição dos dados ao usuário.
- ✓ Controller (Controlador) – Processa entradas do usuário e atualiza a View e o Model.

A interface gráfica de um sistema MVC é composta pelo Controller e pela View.

Interface Gráfica no Padrão MVC

O Papel da View (Vista)

A View é a camada responsável pela apresentação dos dados ao usuário.

Características da View:

- ✓ Exibe informações vindas do Model.
- ✓ Pode ser HTML, telas de aplicativos, interfaces gráficas (GUI).
- ✓ Não contém lógica de negócio.
- ✓ Atualizada pelo Controller quando há mudanças no Model.

Exemplo: Em um sistema web, a View pode ser um HTML com CSS e JavaScript, exibindo os dados recuperados de um banco de dados.

Interface Gráfica no Padrão MVC

O Papel do Controller (Controlador)

O Controller é responsável por gerenciar as interações do usuário e manipular o Model.

Características do Controller:

- ✓ Captura ações do usuário (cliques, envios de formulários, comandos).
- ✓ Atualiza o Model quando necessário.
- ✓ Atualiza a View para refletir as mudanças.
- ✓ Atua como intermediário entre Model e View.

Exemplo: Quando um usuário preenche um formulário e clica em "Salvar", o Controller recebe a ação, atualiza o Model e redireciona para a View correta.

Interface Gráfica no Padrão MVC

Como Controller e View Formam a Interface Gráfica

A interface gráfica no padrão MVC é formada pela interação entre Controller e View:

- ✓ O Controller recebe os eventos do usuário e decide como reagir.
- ✓ A View apresenta os dados e reage às mudanças enviadas pelo Controller.
- ✓ O Model armazena os dados e fornece informações para a View.

MVC

O **MVC** considera três papéis. O **modelo** é um objeto que representa alguma informação sobre o domínio. É um objeto não-visual contendo todos os dados e comportamentos que não são usados pela interface de usuário.

Na sua forma **OO (orientada a objeto) mais pura**, o modelo é um objeto dentro de um **Modelo de Domínio**.

Você também poderia pensar em um **Roteiro de Transação** como o modelo, desde que ele não contenha nenhum mecanismo de interface com o usuário. Tal definição amplia a noção de modelo, mas se adapta à divisão de papéis do **MVC**.

MVC

A **vista** é responsável por exibir as informações, o **controlador** manipula a entrada do usuário e atualiza o modelo, e o **modelo** contém os dados e a lógica de negócios.

A **vista** representa a **exibição do modelo** na interface com o usuário. Assim, se nosso modelo for um objeto cliente, nossa vista poderia ser um **frame cheio de controles** para a interface com o usuário ou uma **página HTML com informações do modelo**.

A vista diz respeito apenas à **apresentação de informações**, sendo que quaisquer alterações nessas informações são manipuladas pelo terceiro membro da tríade **MVC: o controlador**.

MVC

O **controlador** recebe a **entrada do usuário**, manipula o **modelo** e faz com que a **vista seja atualizada** apropriadamente. Dessa forma, a interface de usuário é uma combinação da **vista** e do **controlador**.

A **classe Controller (Controlador)** mais a **classe View (Vista)** constituem a **interface gráfica de sistemas MVC**, garantindo a separação entre a lógica da aplicação (modelo) e a apresentação (vista), controlada pelas interações do usuário (controlador).

Arquitetura em camadas

A arquitetura em camadas é um padrão de design de software que separa as responsabilidades do sistema em diferentes camadas independentes. Cada camada tem um papel específico e se comunica apenas com as camadas adjacentes. Esse modelo melhora a organização do código, facilita a manutenção e permite escalabilidade.

Princípios da Arquitetura em Camadas

- ✓ Separação de responsabilidades: Cada camada tem uma função bem definida.
- ✓ Baixo acoplamento: As camadas se comunicam de forma estruturada, reduzindo dependências diretas.
- ✓ Facilidade de manutenção: Alterações em uma camada não afetam outras diretamente.
- ✓ Maior escalabilidade: Suporte a diferentes implementações sem impactar o sistema inteiro.

Arquitetura em Camadas: Exemplo duas camadas

Estação cliente faz acesso direto ao servidor de banco de dados.

Modelo de 2 camadas (Client-Server)

Neste modelo, o cliente se comunica diretamente com o banco de dados sem intermediários.

Toda a lógica da aplicação pode estar no próprio cliente.

Exemplo: Aplicações desktop antigas, onde um programa acessa o banco de dados diretamente.

Arquitetura em Camadas: Exemplo duas camadas

Um conjunto de bibliotecas, localizadas no computador cliente, tem a função de viabilizar a comunicação entre ele e o servidor.

Modelo de 2 camadas

Também faz parte da arquitetura cliente-servidor, onde o cliente contém bibliotecas que possibilitam a comunicação direta com o banco de dados.

Pode ser um driver ODBC ou bibliotecas específicas para acessar o banco, como JDBC em Java.

Arquitetura em Camadas: Exemplo três camadas

As conexões no banco de dados são realizadas pelo servidor de aplicação.

Modelo de 3 camadas (Three-Tier)

Aqui, o cliente não acessa diretamente o banco de dados, mas sim um servidor de aplicação, que intermedeia as conexões.

O servidor de aplicação trata a lógica de negócio e acessa o banco conforme necessário.

Exemplo: Aplicações web modernas, como sistemas desenvolvidos em Java EE, .NET ou Node.js.

Arquitetura em Camadas: Exemplo três camadas

Quando é possível ter a mesma regra de negócio dividida entre vários servidores através do balanceamento de carga.

Modelo de 3 camadas (Three-Tier ou Multi-Tier avançado)

Aqui, a lógica de negócio pode ser distribuída entre vários servidores, permitindo escalabilidade.

Essa divisão reduz gargalos de entrada e saída de dados (I/O) e melhora a performance do sistema.

É usada em grandes aplicações corporativas e sistemas escaláveis na nuvem.

Arquitetura de três camadas (three-tier), sobre a qual é correto afirmar:

Uma de suas vantagens é permitir que qualquer uma das três camadas sejam atualizadas ou substituídas de forma independente em resposta a mudanças nos requisitos ou na tecnologia utilizada.

Exemplo:

Em uma empresa, foi contratada uma equipe de TI para implementar o software utilizando um modelo de arquitetura multi-tier. Pesquisando sobre o assunto, um integrante da equipe encontrou a seguinte descrição:

"Em engenharia de software, arquitetura multi-tier é uma arquitetura cliente-servidor em que apresentação, processamento e funções de gerenciamento de dados são separados logicamente. A arquitetura multi-tier mais utilizada hoje é a arquitetura de três camadas (three-tier).“ Adotou-se, então, a arquitetura de três camadas (**three-tier**).

Arquitetura em Multicamadas (N-Tier)

Expansão da arquitetura Three-Tier, onde há mais camadas especializadas, como camada de segurança, serviços, cache, API Gateway, entre outras.

Usada em grandes sistemas distribuídos e aplicações em nuvem.

Exemplo:

Amazon, Google e Facebook usam arquitetura multicamadas para distribuir cargas e otimizar processamento.

Arquitetura em Multicamadas (N-Tier)

Expansão da arquitetura Three-Tier, onde há mais camadas especializadas, como camada de segurança, serviços, cache, API Gateway, entre outras.

Usada em grandes sistemas distribuídos e aplicações em nuvem.

Exemplo:

Amazon, Google e Facebook usam arquitetura multicamadas para distribuir cargas e otimizar processamento.